Lecture #17 | Web access

SE377 Introduction to Big Data Analysis and Visualization (2017)

Prof. Min-gyu Cho

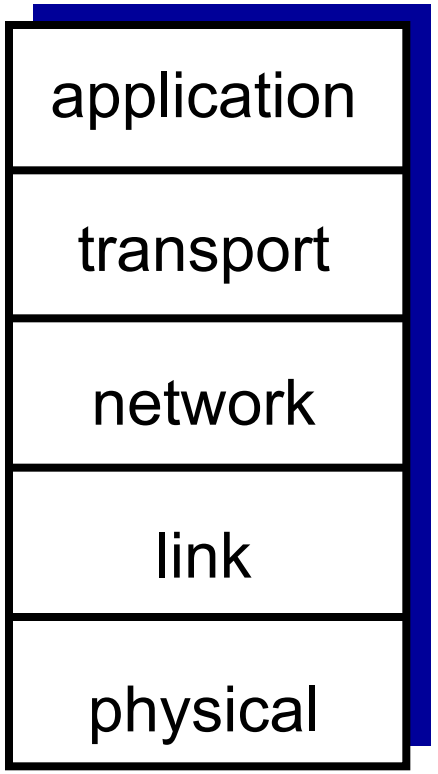
Today's topic

- Web
  - Protocol: Hypertext transport protocol (HTTP)
  - Languages: ***html***, css, javascript

* or 15? 20? 30? 75?

Recap: internet protocol stack & URL

- application: supporting network applications
  - FTP, SMTP, HTTP
- transport: process-process data transfer
  - TCP, UDP
- network: routing of datagrams from source to destination
  - IP, routing protocols
- link: data transfer between neighboring network elements
  - Ethernet, 802.11 (WiFi), PPP
- physical: bits “on the wire”
- Example of URL: http://www.dgist.ac.kr/site/dgist/menu/1.do



HyperText Transfer Protocol (HTTP)

- Simply speaking, HTTP is the protocol for the web
- Or… HTTP is an application protocol for distributed, collaborative, and hypermedia information systems (i.e., web)
- Typically use port 80, but occasionally other ports are used such as 8888, 8080
- Adoption of HTTP secure (HTTPS) is dramatically increasing
  - HTTPS encrypts HTTP messages with transport layer security (TLS) or secure sockets layer (SSL) protocols
  - You can see lock icon in the address bar of modern browsers
  - Public key infrastructure (PKI) plays an important role, which is based on public key cryptography such as RSA and DSA

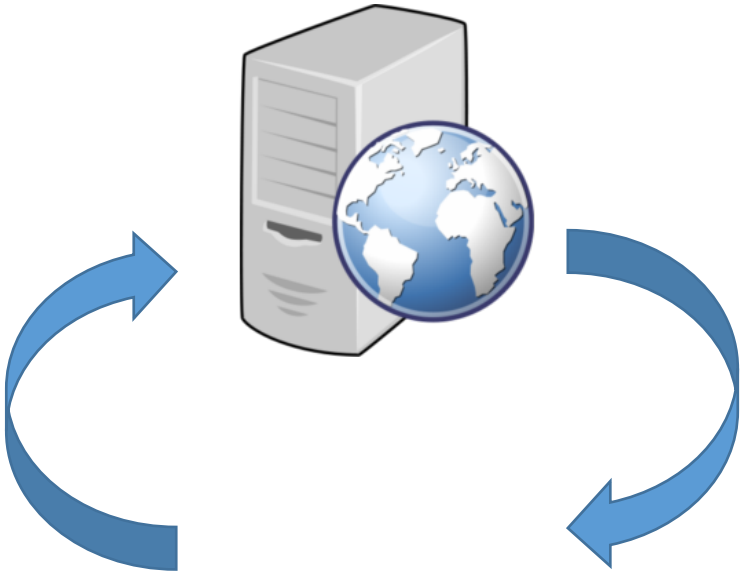
Languages for web client

- Hypertext Markup Language (html)
  - provide contents of web pages
- Cascading Style Sheets (css)
  - specify presentation of web pages such as layout, font (size, color, etc.)
- JavaScript (js)
  - define dynamic behaviors of web pages
- Note: server side languages
  - javascript (node.js), Java, python, ruby, etc.

Example: HTTP protocol in action

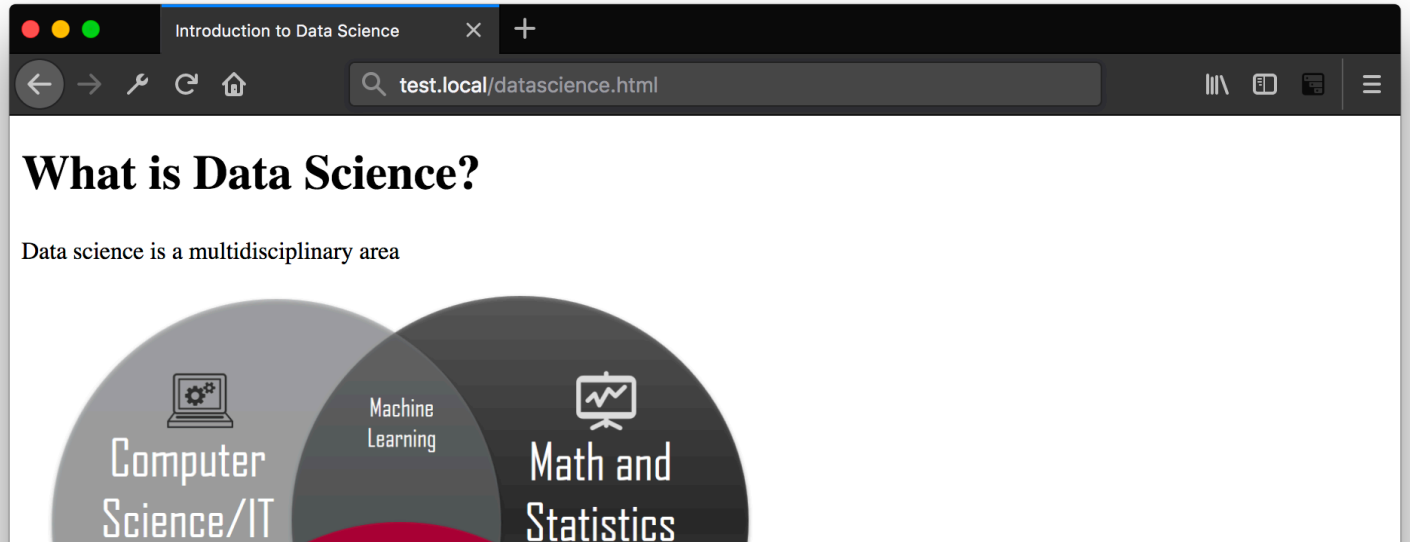
Request

GET
http://test.local/datascience.html

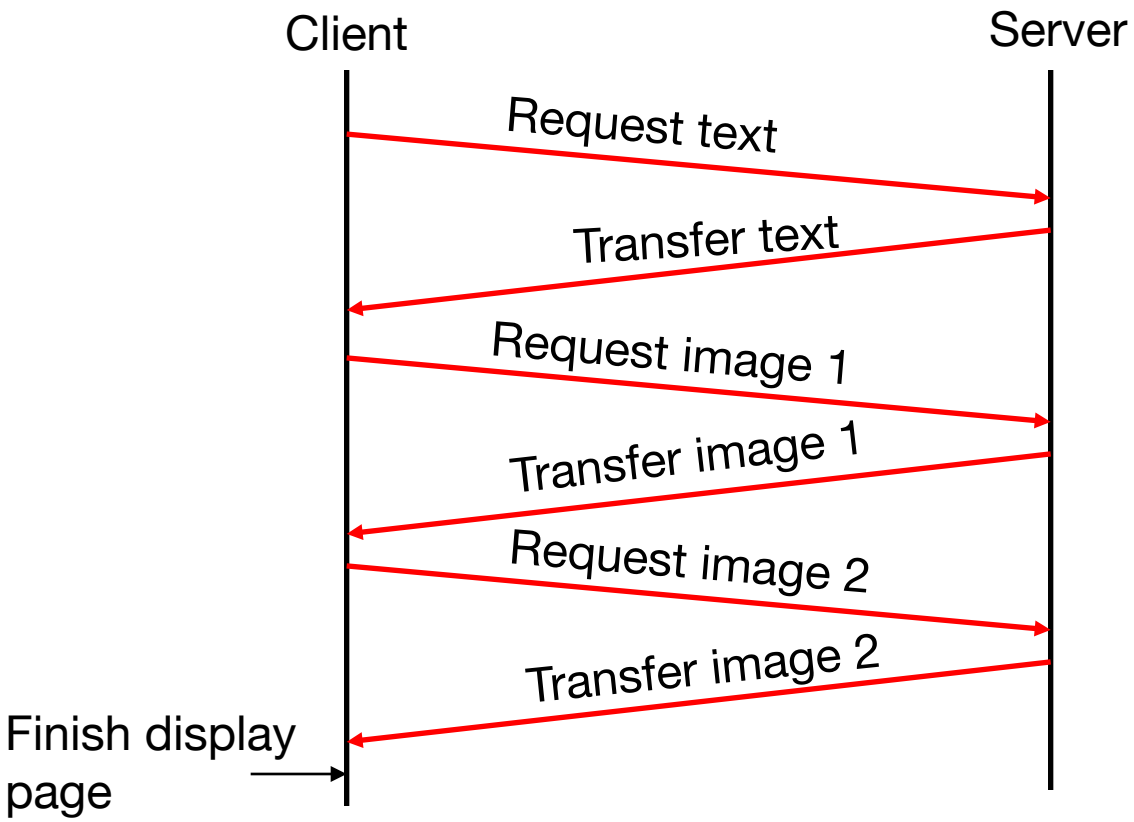


Response

```
<!DOCTYPE HTML>
<html>...
</html>
```



Example: HTTP protocol in action (cont.)



A simple html page

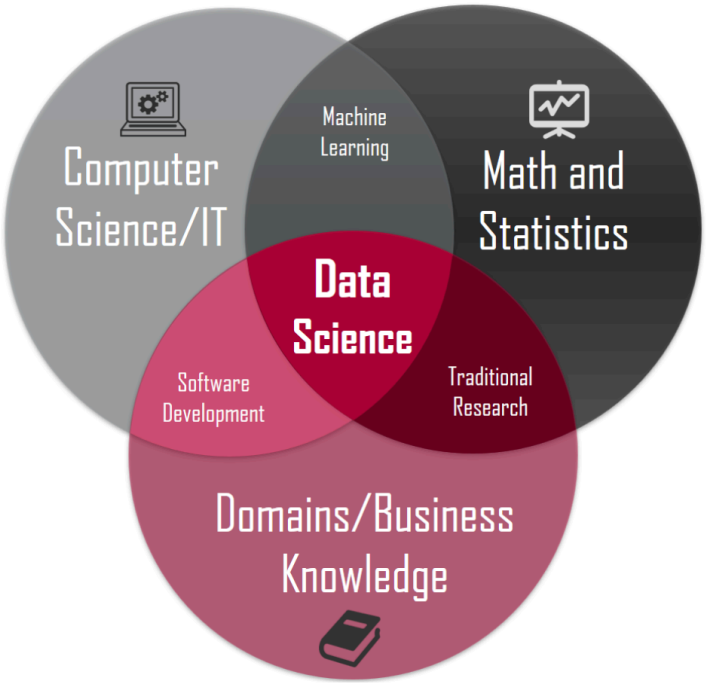
```html
<!DOCTYPE HTML>
<html>
 <head>
  <title>Introduction to Data Science</title>
 </head>
 <body>
  <h1>What is Data Science?</h1>
  <p>Data science is a multidisciplinary area</p>
  <img width="500" src="http://i.imgur.com/nFXKnl7.png">
  <h1>See also</h1>
  <ul>
   <li><a href="https://github.com/chomg/se377_2017">SE377 course page</a></li>
   <li><a href="https://www.kaggle.com">Kaggle</a></li>
  </ul>
 </body>
</html>
```

Introduction to Data Science

test.local/datascience.html

What is Data Science?

Data science is a multidisciplinary area



See also

- SE377 course page
- Kaggle

Request: client-to-server communication

- HTTP Request Message
  - Request line: method, resource, and protocol version
  - Request headers: provide information or modify request
  - Body: optional data (e.g., to “POST” data to the server)

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language: ko
(blank line)
```

Request: client-to-server communication (cont.)

- HTTP Request Message
  - Request line: method, resource, and protocol version
  - Request headers: provide information or modify request
  - Body: optional data (e.g., to “POST” data to the server)
- Request methods include
  - GET: Return current value of resource, run program, …
  - HEAD: Return the meta-data associated with a resource
  - POST: Update resource, provide input to a program, …
- Headers include
  - Useful info for the server (e.g., desired language)

Response: server-to-client communication

- HTTP Response Message
  - Status line: protocol version, status code, status phrase
  - Response headers: provide information
  - Body: optional data (e.g., html, jpg)

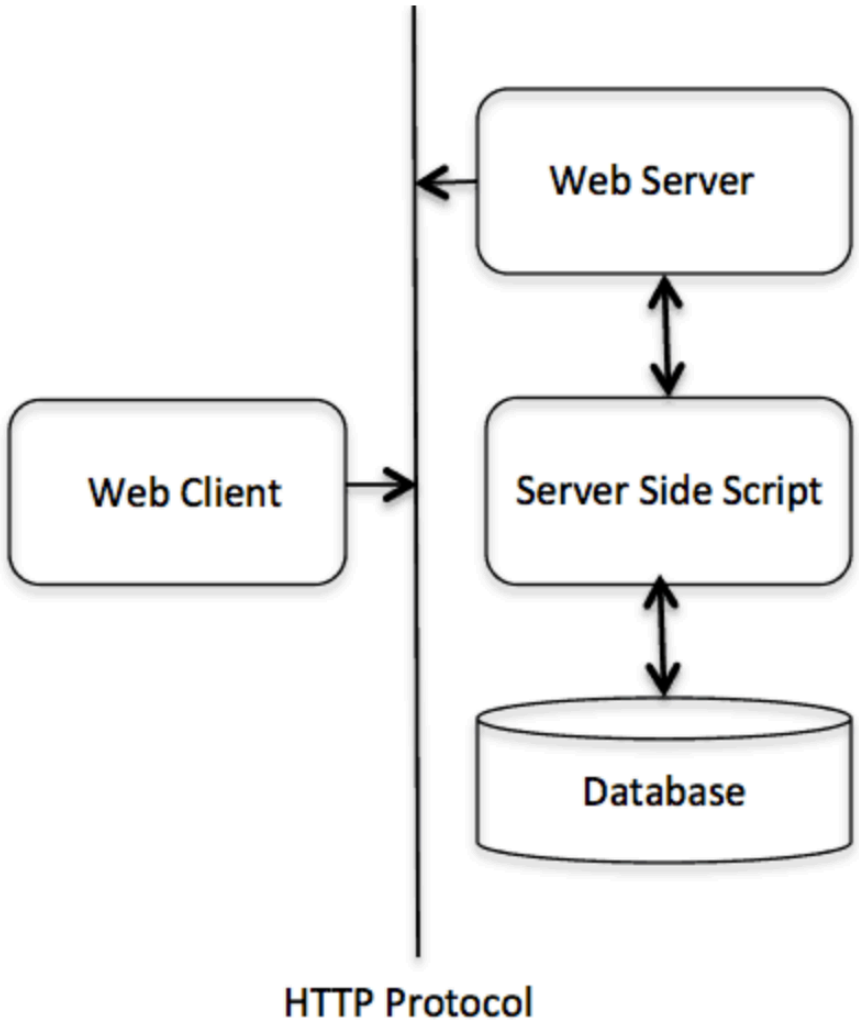
```
HTTP/1.1 200 OK
Connection close
Date: Thu, 06 Aug 2006 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 2006 ...
Content-Length: 6821
Content-Type: text/html
(blank line)
data data data data data ...
```

Response: server-to-client communication (cont.)

- HTTP Response Message
  - Status line: protocol version, status code, status phrase
  - Response headers: provide information
  - Body: optional data
- Response code classes
  - Similar to other ASCII app. protocols like SMTP

Code	Class	Example
1xx	Informational	100 Continue
2xx	Success	200 OK
3xx	Redirection	304 Not Modified
4xx	Client error	404 Not Found
5xx	Server error	503 Service Unavailable

Who generates the response?



Web server: generating a response

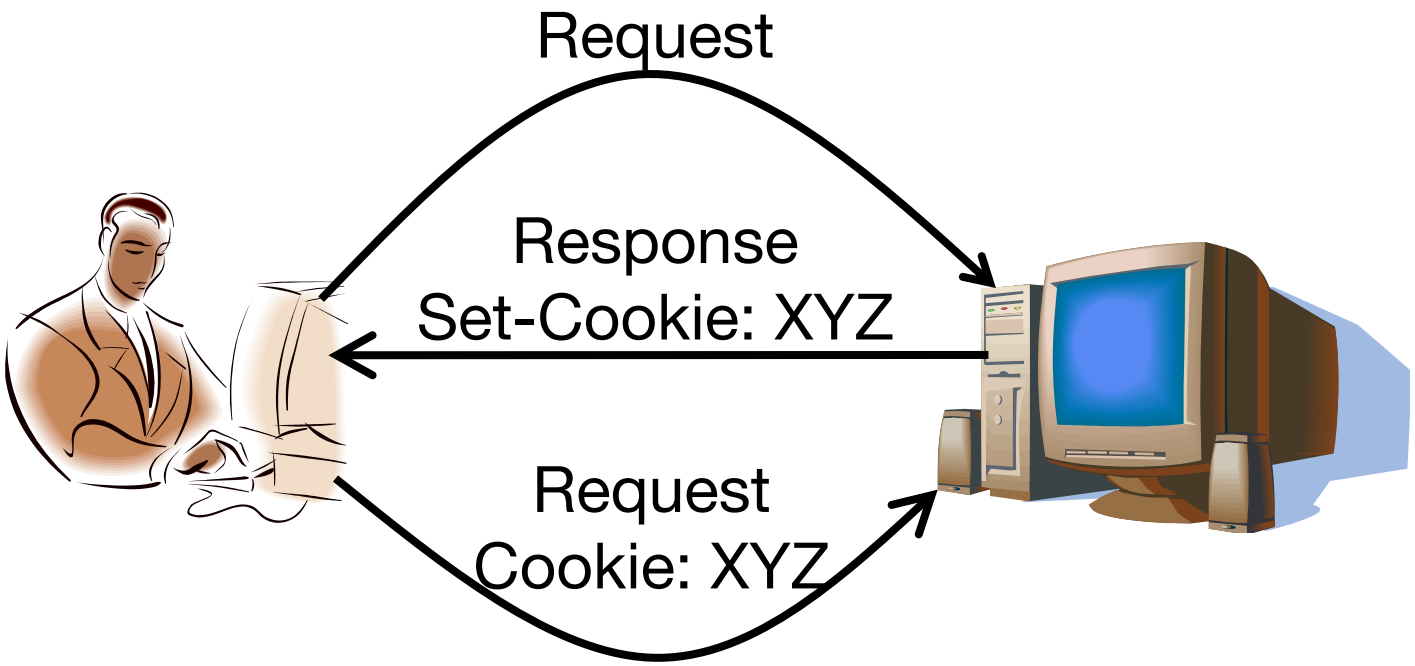
- Types of response
  - Return a file
    - URL matches a file (e.g., /www/index.html)
    - Server returns file as the response
    - Server generates appropriate response header
  - Generate response dynamically
    - URL triggers a program on the server
    - Server runs program and sends output to client
  - Return meta-data with no body
- In real systems, large number of web servers are used for scalability and availability, backed by DB servers, files servers, etc.

HTTP is stateless

- Stateless protocol
  - Each request-response exchange treated independently
  - Servers not required to retain state
- This is good: Improves scalability on the server-side
  - Don’t have to retain info across requests
  - Can handle higher rate of requests
  - Order of requests doesn’t matter
- This is bad: Some applications need persistent state
  - Need to uniquely identify user or store temporary info
  - e.g., Shopping cart, user preferences and profiles, usage tracking, …

Cookies: provides state

- Client-side state maintenance
  - Client stores small(?) state on behalf of server
  - Client sends state in future requests to the server
- Can provide authentication



Web browser: typical client

- Generates HTTP requests
  - User types URL, clicks a hyperlink or bookmark, clicks “reload” or “submit”
  - Automatically downloads embedded images
- Submits the requests (fetches content)
  - Via one or more HTTP connections
- Presents the response
  - Parses HTML and renders the Web page
  - Invokes helper applications (e.g., Acrobat, MediaPlayer)
- Maintains cache
  - Stores recently-viewed objects and ensures freshness

Other types of clients

- Web crawling/scraping engine
  - Web scraper behaves like web browsers, e.g., generates request, store/send cookies
- Mobile apps
  - Majority of mobiles apps communicate with servers
  - They often use HTTP with servers

A structure of html file

```
<!DOCTYPE html>
<html>
  <head>
    <title>This is document title</title>
  </head>
  <body>
    <h1>This is a heading</h1>
    <p>Document content goes here.....</p>
   </body>
</html>
```

doctype

head

body

html

html tag

```
<tagname>content goes here...</tagname>
```

- html tags are usually paired, meaning they appear with a start (or opening) and an end (or closing) tag
- An end tag has the same name but starts with slash
- Note
  - Don’t forget the end tag!
  - Lower case is recommended for tags

html attributes

- All HTML elements can have attributes
- Attributes provide additional information about an element
- Attributes are always specified in the start tag
- Attributes usually come in name/value pairs like
  - `name="value"`
- Examples
  - `<img width="500" src="http://i.imgur.com/nFXKnl7.png">`
  - `<a href="https://github.com/chomg/se377_2017">SE377 course page</a>`

Common html tags

- Headings: `<h1>, <h2>, …, <h6>`
- Paragraph: `<p>`
- Links: `<a>`
  - `href`: the link (typically URL)
- Images: `<img>`
  - `src`: source of image (typically URL)
  - `alt`: alternative text

Additional materials

- General documents on web scraping (with python)
  - 5 tasty python web scraping libraries
  - 7 Best Web Scraping Software tools to Acquire Data Without Coding
- Tutorials on several web languages: https://www.w3schools.com/
- List of html/css tutorials
  - https://www.quora.com/What-is-the-best-way-for-a-beginner-to-learn-HTML-CSS

ANY QUESTIONS?